

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for

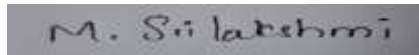
caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Code of Conduct:

I **M. Srilakshmi (192525404)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Q1. Transfer Learning Strategy for Skin Cancer Detection

A suitable approach is to use a **pre-trained ResNet50 CNN** trained on ImageNet and adapt it for classifying dermoscopic images as **benign or malignant**.

1. Choice of Pre-trained Model

ResNet50 is a good choice because:

- It is a deep CNN with strong image-feature extraction capability.
- Its residual connections help avoid the vanishing-gradient problem.
- The pre-trained layers can recognize general features such as edges, textures, shapes, and patterns.
- It performs well even when the target dataset is relatively small.

2. Transfer Learning Architecture

Dermoscopic Image → ResNet50 → Feature Extraction → Fully Connected Layer → Softmax/Sigmoid → Skin Cancer Class

The final classification layer of ResNet50 is removed and replaced with a new layer suitable for the skin-cancer classes.

3. Layers to Freeze and Fine-Tune

Initially freeze most of the ResNet50 layers:

- Freeze the early and middle convolutional layers.
- These layers contain general features such as edges, curves, textures, and basic patterns.
- This reduces the number of trainable parameters and prevents overfitting because the dataset is small.

Fine-tune the later layers:

- Unfreeze the last few convolutional blocks of ResNet50.
- Train these layers using the dermoscopic images.
- The later layers learn more specific features such as lesion shape, pigmentation, irregular borders, and texture patterns.

Replace the final classification layer:

- Remove the original ImageNet classification layer.
- Add **Global Average Pooling → Dense layer → Dropout → Output layer**.
- For binary classification, use a **sigmoid** output.
- For multiple skin-lesion classes, use **softmax**.

4. Data Augmentation

Since labeled medical images are limited, augmentation can increase the effective training data. Suitable techniques include:

- Rotation
- Horizontal/vertical flipping
- Zooming
- Cropping
- Small translations
- Brightness/contrast adjustments

Care should be taken not to apply transformations that significantly change clinically important lesion characteristics.

5. Training Strategy

1. Resize dermoscopic images to the input size required by ResNet50.
2. Normalize the images using the appropriate preprocessing for the pre-trained model.
3. Load the pre-trained ResNet50 without its original classification head.
4. Freeze the majority of the convolutional layers.
5. Add a new classification head.
6. Train the new classification layers first.
7. Unfreeze the final few ResNet50 blocks.
8. Fine-tune them using a **small learning rate**.
9. Use validation data and early stopping to reduce overfitting.

6. How It Improves Performance

Transfer learning improves performance because the model does **not need to learn all visual features from the limited skin-cancer dataset from scratch**. The pre-trained CNN already contains useful low-level and mid-level image features. Fine-tuning the later layers allows these features to become specialized for dermoscopic images.

Therefore, the proposed strategy provides:

- Better classification accuracy with limited labeled data.
- Faster training.
- Reduced computational requirements.
- Lower risk of overfitting compared with training a CNN from scratch.
- Better ability to identify complex visual patterns associated with skin cancer.

Final Design

Pre-trained ResNet50 → Freeze early layers → Fine-tune final convolutional blocks → Add new classification head → Train on dermoscopic images → Benign/Malignant prediction

This makes ResNet50-based transfer learning with selective fine-tuning an effective strategy for developing a skin-cancer detection system when only a limited number of labeled dermoscopic images are available.

Q2. DenseNet and PixelNet – Road-Scene Analysis

A suitable solution is a **DenseNet–PixelNet based semantic segmentation architecture**. The system takes a road-scene image as input and produces a pixel-level segmentation map identifying **road, pedestrians, vehicles, traffic signs, and other objects**.

1. Proposed Architecture

Input Road Image → DenseNet Encoder → PixelNet Feature Extraction → Multi-scale Feature Fusion → Pixel-wise Classifier → Segmentation Map

2. DenseNet as Feature Extractor

Use a pre-trained **DenseNet-121** as the encoder.

DenseNet connects each layer to every subsequent layer within a dense block:

Layer 1 → Layer 2 → Layer 3 → Layer 4 → ...

Each layer receives the feature maps generated by all previous layers.

This provides:

- **Feature reuse:** Earlier features such as edges and textures are reused by deeper layers.
- **Better gradient flow:** Direct connections make training deep networks easier.
- **Fewer redundant features:** Features are reused instead of repeatedly learning similar representations.
- **Strong feature extraction:** DenseNet can capture both low-level and high-level road-scene information.

3. PixelNet for Pixel-Level Prediction

The extracted DenseNet features are given to a **PixelNet-style pixel-level prediction module**.

PixelNet treats segmentation as a **pixel-wise classification problem**. For every pixel, the network predicts the corresponding class:

- Road

- Pedestrian
- Vehicle
- Traffic sign
- Building
- Background

The PixelNet module combines features from different resolutions so that both object-level and detailed pixel-level information are preserved.

4. Multi-Scale Feature Fusion

Features from different DenseNet blocks are combined before classification.

For example:

Low-level features:

Edges, boundaries and textures

Middle-level features:

Vehicle shapes, pedestrian structures and signs

High-level features:

Complete objects and scene context

These features are fused to produce a detailed segmentation representation.

5. Pixel-Wise Classification

A final 1×1 **convolutional layer** converts the extracted features into class scores for every pixel.

For example:

Input Image



DenseNet-121 Encoder



Dense Blocks



Multi-scale Feature Fusion



PixelNet Pixel-level Module



1×1 Convolution



Softmax



Pixel-wise Segmentation Map

The output is an image where every pixel is assigned to a particular road-scene category.

6. Improving Segmentation Accuracy

The architecture improves segmentation accuracy because:

- DenseNet preserves information from earlier layers.
- Multi-scale features help detect both small and large objects.
- Pixel-level prediction provides accurate object boundaries.
- High-level features provide contextual information.
- Low-level features help preserve fine details such as pedestrian boundaries and traffic-sign edges.

A suitable segmentation loss can be **cross-entropy loss**, optionally combined with **Dice loss** or **IoU-based loss** to improve performance on small objects such as pedestrians and traffic signs.

7. Computational Efficiency

DenseNet improves computational efficiency through **feature reuse**. Instead of learning completely new feature representations at every layer, subsequent layers reuse existing features.

Additional efficiency can be achieved by:

- Using DenseNet-121 instead of a much larger backbone.
- Applying bottleneck layers using **1×1 convolutions**.
- Reducing feature-map dimensions using transition layers.
- Processing selected multi-scale features rather than all intermediate features.
- Using GPU acceleration during training and inference.

8. Overall Advantages

Requirement	DenseNet–PixelNet Solution
Feature reuse	Dense connections reuse previous features
Road detection	High-level semantic features
Pedestrian detection	Multi-scale and detailed features

Requirement	DenseNet–PixelNet Solution
Vehicle detection	Strong object-level representations
Traffic-sign detection	Fine-resolution pixel features
Boundary accuracy	Low-level features preserve edges
Computational efficiency	Dense feature reuse and bottleneck layers
Pixel-level output	PixelNet-style pixel classifier

Conclusion

The proposed DenseNet–PixelNet architecture uses DenseNet as an efficient feature-extraction backbone and PixelNet for pixel-level classification. Dense connections provide effective feature reuse and better gradient propagation, while PixelNet combines multi-scale information for accurate pixel-wise segmentation. As a result, the system can accurately identify roads, pedestrians, vehicles, and traffic signs while maintaining good computational efficiency, making it suitable for automated smart-city road-scene analysis.

Q3. Bidirectional RNN and Encoder–Decoder for Multilingual Chatbot

A suitable solution is a **Bidirectional RNN-based Encoder–Decoder sequence-to-sequence model**. The model receives a customer query in one language, understands its complete context, and generates the translated query in another language.

1. Proposed Architecture

Customer Query



Tokenization & Word Embedding



Bidirectional RNN Encoder



Forward + Backward Hidden States



Context Representation



Attention Mechanism



RNN Decoder



Target Language Tokens



Translated Customer Query

2. Bidirectional RNN Encoder

The input sentence is first converted into word/token embeddings.

For example:

English:

"I want to cancel my flight"

The **Bidirectional RNN** processes the sentence in two directions:

- **Forward RNN:** processes words from left → right.
- **Backward RNN:** processes words from right → left.

The hidden states from both directions are combined:

where:

- = forward hidden state
- = backward hidden state
- = combined contextual representation

Thus, each word receives information from both its **previous and following words**.

3. Encoder–Decoder Structure

The **encoder** reads the complete source-language sentence and converts it into contextual representations.

The **decoder** then generates the translated sentence one word at a time.

For example:

English Input:

"I want to cancel my flight"



Bi-RNN Encoder



Context + Attention



RNN Decoder



French Output:

"Je veux annuler mon vol"

The decoder uses the previously generated word and the encoder's contextual information to predict the next word.

4. Attention Mechanism

Instead of relying only on one fixed context vector, an **attention mechanism** allows the decoder to focus on the most relevant encoder states while generating each output word.

For example, when generating the translation of "**flight**", the decoder can give greater attention to the corresponding source word and its surrounding context.

This is particularly useful for long customer queries.

5. Why Bidirectional Processing Improves Translation

A standard RNN encoder mainly processes information in one direction. A Bidirectional RNN considers information from **both sides of each word**.

For example:

"I saw a **bank** near the river."

The word "**bank**" can have different meanings. The backward context containing "**near the river**" helps the network understand that it refers to a financial/river-bank context rather than another possible meaning.

Therefore, bidirectional processing:

- Captures complete sentence context.
- Resolves word ambiguity.
- Understands relationships between distant words.
- Improves grammatical and semantic understanding.
- Produces more accurate translations.
- Helps preserve the meaning of customer queries.

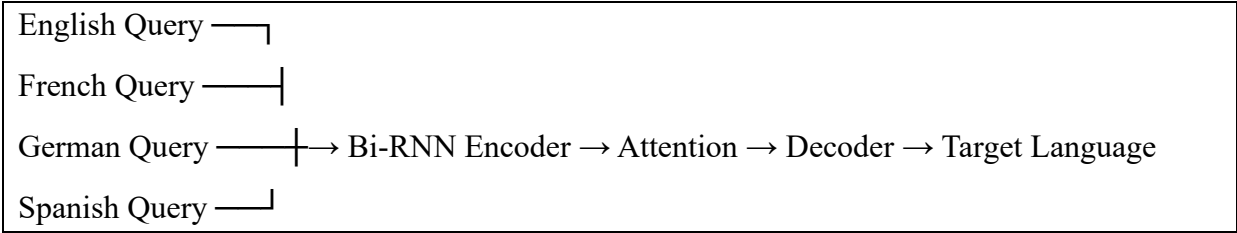
6. Multilingual Extension

For a multilingual chatbot, the same architecture can support multiple languages by using:

- Language-specific tokenization.
- Shared or multilingual word embeddings.
- A language identifier such as <EN>, <FR>, <DE>, etc.

- A decoder trained to generate the required target language.

For example:



7. Advantages

Component	Role
Word Embedding	Converts words into numerical vectors
Bi-RNN Encoder	Captures past and future context
Attention	Focuses on relevant source words
RNN Decoder	Generates translated sequence
Softmax Layer	Selects the next target-language word
Language Token	Specifies the required target language

Conclusion

The proposed Bidirectional RNN Encoder–Decoder with Attention is well suited for a multilingual customer-service chatbot. The Bidirectional RNN captures contextual information from both directions, while the attention mechanism allows the decoder to focus on relevant parts of the input sentence. This combination improves semantic understanding, handling of ambiguous words, long-range dependencies, and overall translation accuracy, resulting in more meaningful multilingual customer responses.

Q4. BPTT and LSTM for Stock Price Prediction

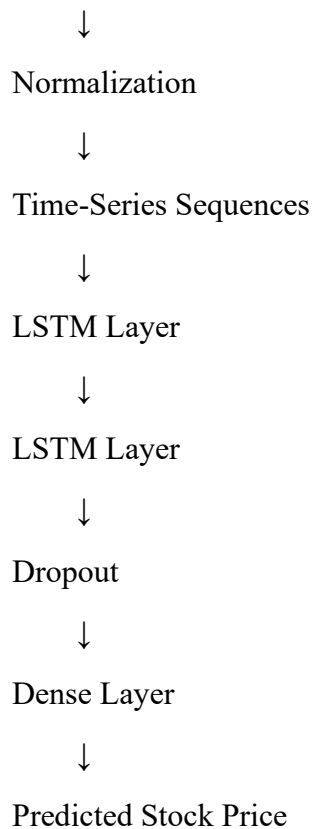
A suitable solution is an **LSTM-based time-series prediction system** that uses historical stock-market data to predict future stock prices. LSTM is preferred over a traditional RNN because it can learn long-term dependencies in sequential financial data.

1. Proposed Architecture

Historical Stock Data



Data Preprocessing



The input data can contain features such as:

- Opening price
- Closing price
- High price
- Low price
- Trading volume
- Moving averages
- Technical indicators

For example, the previous **60 trading days** can be used to predict the stock price for the next trading day.

2. LSTM Network

An LSTM contains three important gates:

Forget Gate:

Determines which previous information should be discarded.

Input Gate:

Determines which new information should be stored in the cell state.

Output Gate:

Determines which information should be passed to the next time step.

The basic LSTM operations are:

Here, is the cell state and is the hidden state.

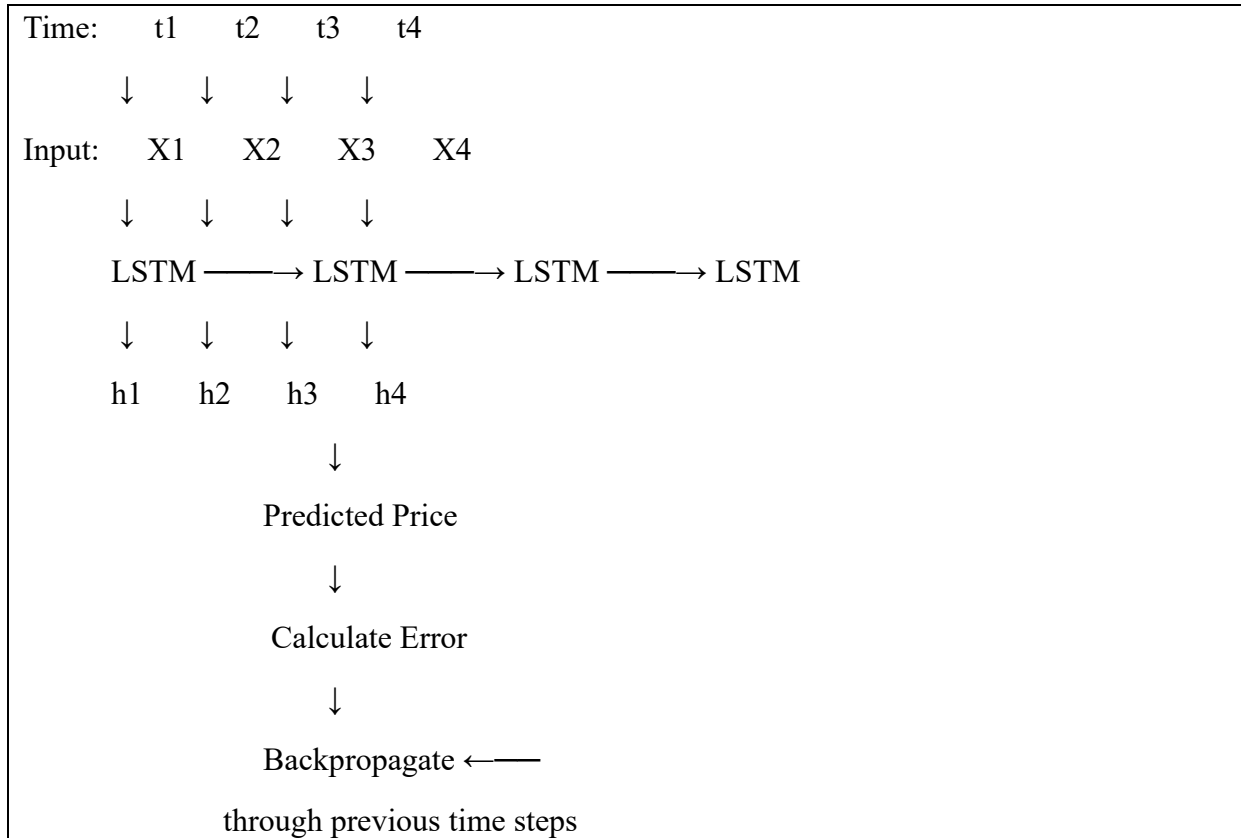
3. Backpropagation Through Time (BPTT)

BPTT is used to train the LSTM by calculating how the prediction error should modify the network's parameters across the sequence of time steps.

The process is:

1. Historical stock data is divided into time sequences.
2. The sequence is passed through the LSTM one time step at a time.
3. The LSTM generates a predicted stock price.
4. The predicted value is compared with the actual value using a loss function such as **Mean Squared Error (MSE)**.
5. The error is propagated backward through all the time steps.
6. Gradients are calculated for the LSTM weights and gates.
7. An optimizer such as **Adam** updates the weights.
8. This process is repeated for multiple epochs until the prediction error is minimized.

4. BPTT in LSTM



BPTT therefore allows the model to learn how information from **earlier market conditions affects later predictions**.

5. Why LSTM is Better Than Traditional RNN

A traditional RNN has difficulty learning long-term dependencies because repeated multiplication during BPTT can cause **vanishing or exploding gradients**.

For stock-price prediction, relationships may exist across many previous time steps. LSTM addresses this using its **cell state and gating mechanism**.

Traditional RNN	LSTM
Limited long-term memory	Strong long-term memory
Can suffer from vanishing gradients	Gates help preserve useful gradients
Simple hidden-state mechanism	Cell state + three gates
Less effective for long sequences	Better for long time-series sequences
May forget important historical patterns	Can selectively retain important information

6. Advantages for Financial Forecasting

LSTM is particularly useful because it can:

- Learn patterns from historical price sequences.
- Retain important information over long periods.
- Identify temporal relationships between market variables.
- Handle nonlinear time-series patterns.
- Reduce the effect of irrelevant historical information through its forget gate.
- Produce continuous numerical predictions for future prices.

Conclusion

The proposed LSTM-based stock-price prediction system uses historical market features as sequential input, processes them through one or more LSTM layers, and produces the predicted future stock price through a dense output layer. BPTT trains the network by propagating prediction errors backward through the sequence and updating the LSTM parameters. LSTM is more suitable than a traditional RNN because its cell state and gating mechanisms preserve important long-term information and reduce vanishing-gradient problems, making it well suited for financial time-series forecasting.

Q5. Integrated Deep Learning Solution for Automatic Video Caption Generation

A suitable solution is a **Transfer Learning + LSTM Encoder–Decoder architecture**. The system extracts meaningful visual information from video frames using a pre-trained CNN and then uses an LSTM-based sequence model to generate a meaningful caption.

1. Proposed Architecture

Input Video



Video Frame Extraction



Pre-trained CNN

(Transfer Learning)



Visual Feature Extraction



Feature Sequence



LSTM Encoder



Context Representation



LSTM Decoder



Word-by-Word Caption Generation



Final Video Caption

2. Transfer Learning for Visual Feature Extraction

A pre-trained CNN such as **ResNet50**, **InceptionV3**, or **EfficientNet** can be used.

The CNN is originally trained on a large image dataset. Instead of training a CNN from the beginning, its learned visual knowledge is reused.

For example, the video:

"A man is playing football"

is divided into several frames.

The CNN processes each frame and extracts features such as:

- Objects
- People
- Background
- Actions
- Shapes
- Textures
- Scene information

The final classification layer of the CNN is removed, and the output of an intermediate feature layer is used as the **visual feature vector**.

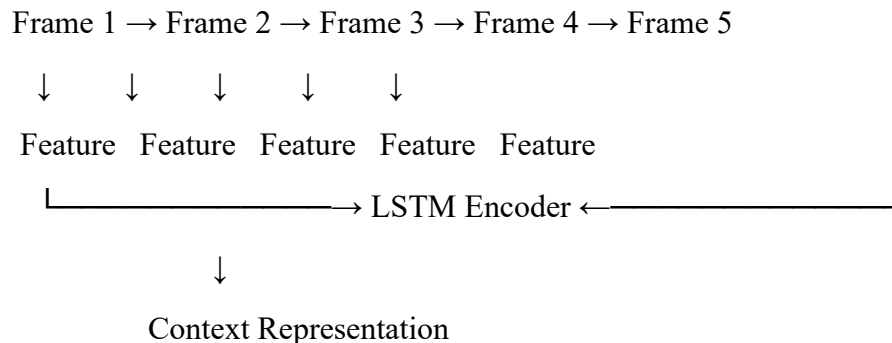
3. LSTM Encoder

The extracted features from multiple video frames form a sequence:

where f_t represents the visual feature of the frame.

The LSTM encoder processes these features sequentially and learns the temporal information in the video.

For example:



This allows the system to understand not only **what objects are present**, but also **what is happening over time**.

4. LSTM Decoder

The decoder converts the encoded visual information into a sequence of words.

It generates the caption one word at a time:

<START>

↓

"A"

↓

"A man"

↓

"A man is"

↓

"A man is playing"

↓

"A man is playing football"

↓

<END>

At every step, the decoder uses:

- Previous generated word
- LSTM hidden state
- Encoded video features

to predict the next word.

5. Attention Mechanism

An **attention mechanism** can be added between the encoder and decoder.

Instead of treating all video frames equally, attention allows the decoder to focus on the most relevant frames when generating each word.

For example:

Frames showing man → "man"

Frames showing ball → "football"

Frames showing action → "playing"

This improves the relationship between visual events and generated words.

6. Training Process

During training, videos with corresponding human-generated captions are provided.

For example:

Input: Video of a person riding a bicycle

Target caption:

"A person is riding a bicycle."

The model predicts the caption and compares it with the actual caption using a suitable sequence loss, commonly **categorical cross-entropy**.

The error is propagated backward through:

LSTM Decoder



LSTM Encoder



Visual Feature Processing



CNN layers being fine-tuned

The weights are updated using an optimizer such as **Adam**.

7. Role of Each Component

Component	Role
Video Frame Extraction	Converts video into manageable image sequences
Pre-trained CNN	Extracts visual features from frames
Transfer Learning	Reuses knowledge learned from a large dataset
LSTM Encoder	Learns temporal relationships between frames
Attention	Focuses on relevant visual information
LSTM Decoder	Converts visual information into words
Softmax Layer	Predicts the probability of the next word
Caption Output	Produces the final natural-language description

8. Why the Integrated System is Effective

Transfer Learning improves performance because the CNN already understands general visual patterns and does not need to learn all image features from scratch.

LSTM Encoder captures temporal information, allowing the system to understand the sequence of events in the video.

LSTM Decoder generates grammatically ordered captions one word at a time.

Attention helps the decoder focus on important frames when generating particular words.

Therefore, the complete system can understand both **spatial information** from individual frames and **temporal information** across the video.

9. Example

Suppose the uploaded video contains:

Frame 1 → A boy

Frame 2 → A boy holding a ball

Frame 3 → A boy throwing the ball

Frame 4 → A boy playing with a dog

The integrated model can generate:

"A boy is playing with a dog and throwing a ball."

The CNN identifies the objects and visual features, while the LSTM learns the sequence of events and generates a meaningful sentence.

Conclusion

The proposed Transfer Learning + LSTM Encoder–Decoder architecture provides an effective solution for automatic video caption generation. Transfer learning extracts rich visual features from video frames, the LSTM encoder captures temporal relationships between frames, and the LSTM decoder converts this information into natural-language captions. Adding attention further improves accuracy by allowing the decoder to focus on relevant video frames. Thus, the integrated system can generate meaningful, context-aware, and grammatically appropriate captions, improving accessibility for users of the media-streaming platform.